Lezione 6

Lezione 6 — Persistenza con un bind mount

In questa esercitazione riprendiamo l'esempio della lezione 5, ma salviamo i dati di MySQL in una cartella del progetto invece di usare un volume nominato. La cartella locale `mysql-data` viene montata nella directory `/var/lib/mysql` del container.

Obiettivi

- usare un bind mount in Docker Compose;
- rendere i dati del database visibili nel sistema operativo;
- distinguere bind mount e volume nominato;
- verificare che i dati sopravvivano alla ricreazione dei container;
- capire la differenza tra `docker compose down` e la rimozione della cartella locale.

Prerequisiti

- Docker Desktop oppure Docker Engine avviato;
- un browser web;
- un terminale aperto nella directory `lezione6`.

Verificare l'installazione:

```
docker --version
docker compose version
```

1. Esaminare la configurazione

La directory contiene il file Compose:

```
lezione6/
├── compose.yaml
└── README.md
```

Nel servizio `db` è presente questo montaggio:

```
volumes:
  - ./mysql-data:/var/lib/mysql
```

La parte a sinistra, `./mysql-data`, è una cartella locale relativa alla directory che contiene `compose.yaml`. La parte a destra, `/var/lib/mysql`, è la directory dei dati di MySQL dentro il container.

Docker creerà la cartella `mysql-data` al primo avvio, se non esiste già.

Controllare la configurazione senza avviare i servizi:

```
docker compose config
```

2. Avviare i servizi

```
docker compose up -d
```

Compose crea automaticamente la rete del progetto, la cartella locale `mysql-data` e i due container.

Controllare lo stato:

```
docker compose ps
```

MySQL potrebbe impiegare alcuni secondi per completare l'avvio. Controllare i log con:

```
docker compose logs -f db
```

Quando compare un messaggio simile a `ready for connections`, premere `Ctrl+C`. I servizi continueranno a funzionare.

3. Collegarsi ad Adminer

Aprire http://localhost:8080 e usare questi valori:

```
Sistema: MySQL
Server: db
Utente: root
Password: rootpass
Database: scuola
```

Il server è `db`, cioè il nome del servizio MySQL nella rete Docker Compose. Non usare `localhost`: dentro Adminer indicherebbe il container Adminer stesso.

4. Creare una tabella e inserire i dati

In Adminer scegliere **SQL command** ed eseguire:

```
CREATE TABLE studenti (
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nome VARCHAR(100) NOT NULL,
    corso VARCHAR(100) NOT NULL
);

INSERT INTO studenti (nome, corso) VALUES
    ('Anna Rossi', 'Docker'),
    ('Luca Bianchi', 'Kubernetes');

SELECT * FROM studenti;
```

Verificare che vengano visualizzati i due studenti.

5. Osservare i dati nel sistema operativo

Dal terminale verificare che la cartella sia stata creata:

```
ls -la
ls -la mysql-data
```

La cartella contiene i file interni usati da MySQL. Non modificarli o rinominarli manualmente mentre il database è in esecuzione.

Per vedere il bind mount dal punto di vista di Docker:

```
docker inspect lezione6-db
```

Nella sezione `Mounts` deve comparire un montaggio con:

```
Destination: /var/lib/mysql
Type: bind
```

6. Verificare la persistenza

Con i dati ancora presenti nel database, rimuovere i container e la rete:

```
docker compose down
```

La cartella locale `mysql-data` non viene rimossa. Verificarne la presenza:

```
ls -la mysql-data
```

Ricreare i servizi:

```
docker compose up -d
docker compose ps
```

Attendere che MySQL sia pronto, accedere nuovamente ad Adminer e verificare che la tabella `studenti` contenga ancora i due record.

I container sono nuovi, ma MySQL riutilizza i dati presenti nella cartella locale montata.

7. Eliminare anche i dati

`docker compose down` non elimina una cartella locale. Per cancellare i dati, prima fermare i servizi:

```
docker compose down
```

Poi rimuovere esplicitamente la cartella `mysql-data` dal sistema operativo. Da macOS o Linux:

```
rm -rf mysql-data
```

Su Windows è possibile cancellarla dal file manager oppure con PowerShell:

```
Remove-Item -Recurse -Force .\mysql-data
```

Al successivo avvio:

```
docker compose up -d
```

MySQL inizializzerà una nuova directory dati e la tabella `studenti` non sarà più presente.

Confronto con la lezione 5

Nella lezione 5 i dati erano salvati in un volume nominato gestito da Docker:

```
- mysql-data:/var/lib/mysql
```

In questa lezione i dati sono salvati direttamente nella directory del progetto:

```
- ./mysql-data:/var/lib/mysql
```

Con un volume nominato, il percorso fisico è gestito da Docker. Con un bind mount, il percorso locale è scelto esplicitamente dall'utente ed è visibile nel file manager.

Riepilogo

- un bind mount collega una cartella locale a una directory del container;
- `./mysql-data` contiene i dati persistenti di MySQL;
- `docker compose down` rimuove container e rete, ma conserva la cartella;
- per eliminare i dati bisogna rimuovere esplicitamente `mysql-data`;
- Adminer continua a raggiungere MySQL usando il nome del servizio `db`.